

AdapTrack: Constrained Decoding without Distorting LLM's Output Intent

Yongmin Li

Key Laboratory of High Confidence Software
Technologies (Peking University), Ministry of Education
Peking University
Beijing, China
liyongmin@pku.edu.cn

Ge Li*

Key Laboratory of High Confidence Software
Technologies (Peking University), Ministry of Education
Peking University
Beijing, China
lige@pku.edu.cn

Jia Li

College of AI
Tsinghua University
Beijing, China
jia_li@mail.tsinghua.edu.cn

Zhi Jin

School of Computer Science
Wuhan University
Wuhan, China
zhijin@whu.edu.cn

Abstract

Language model-based code generation and completion tools have been widely adopted, but they may sometimes produce code that does not meet necessary constraints, such as syntactic correctness or API existence. Constrained decoding techniques are developed to help the model generate code that adheres to the constraints by greedily eliminating generation options that violate constraints at each step of the generation process. However, there is a severe limitation of constrained decoding, that it distorts the model's output intent, forcing it to produce code that may satisfy the constraint but does not match the development intent and is therefore incorrect. In response to this challenge, we propose AdapTrack. By incorporating backtracking into the generation process, AdapTrack avoids distorting the output intent of the language model, thereby producing results that are not only constraint-compliant but also more semantically aligned with model's output intent. On our synthetic API completion dataset, AdapTrack can achieve up to **360.87%** improvement compared to constrained decoding; on the real-world API completion dataset we collect that exhibits similar issues, AdapTrack can also achieve an improvement of up to **38.93%** over constrained decoding; in general code generation benchmarks, compared to constrained decoding, AdapTrack can achieve up to 7.84% improvement on HumanEval, and up to 6.42% improvement on MBPP. This indicates that, simply by better adhering to the model's output intent, AdapTrack can achieve significant improvements. We provide a theoretical proof that the distribution produced by AdapTrack aligns with the model's distribution given the generated tokens, thereby ensuring that the model's output intent is not distorted. Experiments on domain-specific language problems show that, compared to existing methods, our approach

can provide generation results that are more consistent with the language model's distribution. Our code and supplementary material are available at <https://github.com/Gompyn/adaptrack>.

CCS Concepts

• **Software and its engineering** → **Automatic programming**;
• **Computing methodologies** → *Planning under uncertainty*; •
Theory of computation → *Backtracking*.

Keywords

Code Generation, Constrained Decoding, Large Language Models, Distributional Bias

ACM Reference Format:

Yongmin Li, Jia Li, Ge Li, and Zhi Jin. 2026. AdapTrack: Constrained Decoding without Distorting LLM's Output Intent. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773144>

1 Introduction

In recent years, language model-based code generation and completion tools, such as GitHub Copilot [12], have become increasingly popular. These tools can understand the developer's intent from the context and generate code that aligns with their intent by leveraging the capabilities of the language models [2, 9, 18, 20, 23].

Legitimate code must satisfy various constraints, such as adhering to syntax rules and ensuring that the called APIs exist. Language models are designed solely to predict the distribution of the next token [30, 33], but lack built-in mechanisms to guarantee that the generated code complies with these constraints. As a result, the generated code, while potentially aligned with the developer's intent, may sometimes be illegitimate.

Constrained decoding methods [3, 29, 32, 35] are developed to tackle such problems. Using a constrainer to identify tokens illegal to come after already generated text, it simply eliminates the possibility to generate these illegal tokens by directly setting their

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773144>

```
# pytorch 2.0.0
import torch
def matrix_rank(x: torch.Tensor):
    return torch.
```

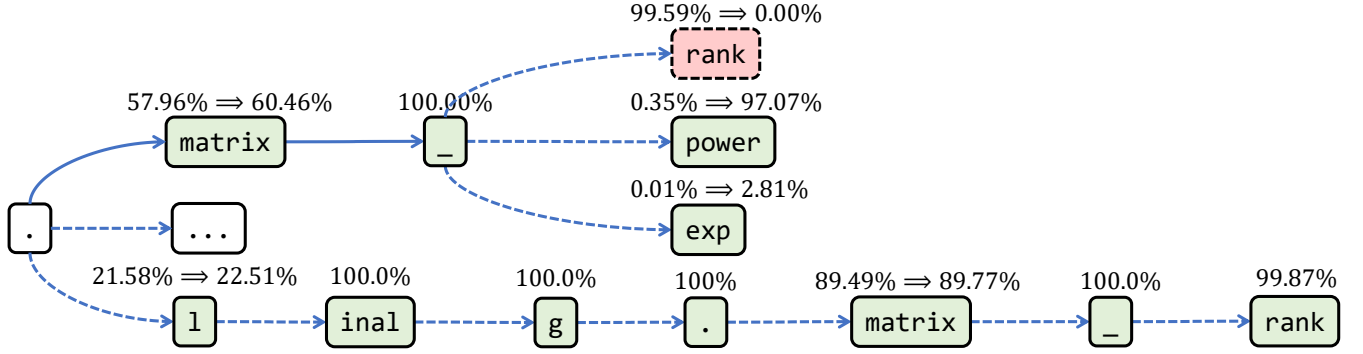
(a) The example code context. We want an API to calculate the rank of the tensor.

candidates	probability	usable
matrix_rank	57.44%	✗
linalg.matrix_rank	19.21%	✓
...		
matrix_power	0.20%	✓

(b) Candidate APIs the model wants to generate, and whether they are usable in PyTorch 2.0.0.

candidates	probability
matrix_power	57.70%
linalg.matrix_rank	20.12%
...	

(c) Candidate APIs in constrained decoding. Compare with Figure 1b.



(d) What happened during the constrained decoding.

Figure 1: An example of incorrect decoding results in constrained decoding.

probability to zero in the model’s output distribution. Such constraints are often available [3, 35], because the Integrated Development Environment (IDE) often provides developers with similar functionality during their development.

However, there is a **severe limitation of constrained decoding** [4, 22, 27], that it distorts the output intent of the model and forces the model to produce unnatural code. In other words, the generated code, while satisfying the constraints, may deviate from the model’s expected functionality and fail to align with the contextual semantics. Figure 1 shows an example to explain this limitation. The input code context is shown in Figure 1a. The user wants to calculate the rank of a matrix using PyTorch [5] 2.0. Two APIs can accomplish this task: `torch.matrix_rank` for PyTorch 1.0 to PyTorch 1.8, and `torch.linalg.matrix_rank` for PyTorch after 1.8. Depending on the user’s PyTorch version, the model needs to select the appropriate API. Figure 1b shows the API names the model intends to output, along with their corresponding probabilities. Its top candidate is `matrix_rank`, and second candidate is `linalg.matrix_rank`. Using a constrainer, it becomes clear that the only legal API in the current version is `linalg.matrix_rank`. Ideally, since we have prohibited the now-illegal `matrix_rank` by constrained decoding, the decoding process should most likely output the most possible candidate under constraint, i.e. `linalg.matrix_rank`. However, as shown in Figure 1c, the API with the highest generation probability in constrained decoding is actually `matrix_power` at 57.70%! This is entirely unexpected because, without constrained decoding, its generation probability is only 0.20%, indicating that the model considers it highly unsuitable for this context. How does this API suddenly become the top choice under constrained decoding?

Let us examine the constrained decoding process step by step, as illustrated in Figure 1d. In the first time step, the legal options include `matrix`, `1`, and several other less possible tokens. Since there

are other valid APIs starting with `matrix`, such as `matrix_power` and `matrix_exp`, the constrainer does not eliminate the possibility of generating `matrix`. As a result, the model still chooses the token `matrix` with a probability of 60.46%. However, once `matrix` and the subsequent `_` token are generated, we find that the token `rank`, which accounts for 99.59% of the model’s generation intent, is prohibited by the constrainer. At this point, since the model has already generated `matrix_`, it is forced to continue under constrained decoding, leading it to select the token `power`, which is entirely misaligned with its output intent. Consequently, even though the model knows that, aside from `matrix_rank`, the most appropriate output here should be `linalg.matrix_rank`, constrained decoding compels it to output `matrix_power` with a 57.70% probability – an answer the model considers highly incorrect.

We observe that, in the process described above, **the model is ultimately forced to select an API that is entirely misaligned with its generation intent**. The constrainer can only identify errors one step at a time, so the `matrix` option remains valid. However, when we realize that the model’s primary intent is thwarted later, we are still compelled to continue generating based on the previously produced tokens, i.e. `matrix_`. As a result, the generated code, while satisfying the constraints, deviates from the model’s original output intent.

If a human were writing this code and discovered that the previously used API `matrix_rank` was no longer available, but recalled having seen the API `linalg.matrix_rank`, they would typically delete the previously written `matrix_` and start over again. Inspired by this behavior, we wonder if a similar mechanism could be introduced into constrained decoding, allowing the model to quickly backtrack to a point where it might have gone wrong and start anew when encountering such difficulties.

Based on this idea, we propose AdapTrack, which, during the constrained decoding process, dynamically **backtracks according to the proportion of invalid options among the current next-step choices**. If this proportion is large, it indicates that the model is unlikely to want to continue generating along this branch and should switch to another branch; otherwise, it suggests that most of the model’s output intent remains on this branch, and we can proceed with generation. In this way, we address the aforementioned issue by dynamically backtracking to avoid generating results that deviate too far from the model’s intent under constraints.

We provide proof for our algorithm, demonstrating that the distribution it produces is identical to the model’s distribution under constraints given the known generation history. This proves that our method effectively avoids the issue of constrained decoding distorting the model’s generation intent, achieving generation that is both constraint-compliant and aligned with the model’s intent.

To demonstrate that AdapTrack does address the issue we raised in Figure 1 – using unfamiliar APIs correctly with a constrainer, we construct a synthetic dataset, TFv1, based on the update in TensorFlow [1] APIs, deliberately prompting the model to use APIs that are unique to TensorFlow v1. These v1 APIs are still usable in TensorFlow v2, except that they have a longer prefix. Since the model is unaware of the TensorFlow version, its output intent inherently includes options for both versions. By using the constrainer, we examine whether the model can generate these APIs compliant with TensorFlow v1 or v2. The results show that, across multiple code language models, AdapTrack significantly improves API generation accuracy compared to naive constrained decoding, with an improvement of up to **360.87%**.

To ensure that these results are not merely effective on this synthetic dataset, we construct a dataset, TFv1Real, based on real-world usage examples of these APIs on GitHub, and ask the model to complete the API given the preceding context. The results on this dataset show that AdapTrack achieves up to **38.93%** improvement over constrained decoding, once again demonstrating the effectiveness of AdapTrack.

To verify this issue’s presence and AdapTrack’s generalizability in constrained decoding of general code generation, we conduct experiments on HumanEval [9] and MBPP [6] with type constrainer [26]. The results show that AdapTrack can achieve up to **7.84%** improvement on HumanEval, and up to **6.42%** improvement on MBPP, showing the generalizability of AdapTrack.

To demonstrate that the generation results of our method are indeed more aligned with the model’s output intent, we conduct experiments on domain-specific language problems, sampling 2000 results for each problem consecutively and comparing them with 2000 results by previous approaches. The results show that the distribution produced by our method has a significantly lower KL divergence from the correct distribution, compared to existing methods, indicating that it better aligns with the actual distribution.

Robustness analysis on temperature and model size shows that AdapTrack works with different hyperparameters. Experiments with limited backtrack distance shows that a certain short-range backtracking is enough on TFv1 and TFv1Real.

In summary, our contributions are as follows.

- We propose AdapTrack, a constrained decoding method that can avoid distortion of the model’s output intent in greedy constrained decoding.
- We prove that AdapTrack’s distribution aligns with the model’s distribution given the generated tokens. Experiments on DSL problems show that the distribution produced by AdapTrack aligns closer with the model’s true intent.
- We construct the TFv1 and TFv1Real datasets to simulate scenarios where the model is unfamiliar with APIs. The results demonstrate that our method significantly improves the accuracy of API generation, with an improvement up to **360.87%** on TFv1 and **38.93%** on TFv1Real. Experiments on HumanEval and MBPP demonstrate the generalizability of AdapTrack in general code generation.

2 Motivating example

We have seen the problem of constrained decoding in Figure 1. The problem occurs when the decoding process eliminates the probability of one API, but fails to generate another highly possible API with a different prefix. We propose to introduce backtracking into the decoding process as a potential solution, and in this section we will delve into a more comprehensive discussion of this approach.

Figure 2 shows step by step how to utilize this intuition on constrained decoding. The arrows between tokens represent the generation order. A solid arrow indicates that the current state of generation includes this step, whereas a dashed arrow signifies other potential generation options. A red dashed box denotes tokens that are invalid in the current context, and a green solid box represents valid tokens. The numbers on the tokens indicate their conditional generation probabilities given the preceding context.

Figure 2a illustrates the scenario where the issue mentioned in Figure 1 occurs. We have generated `matrix_`, and subsequently identified the following token options, their validity, and how much the language model wants to generate them. We observe that the model has a 99.59% probability to generate the **invalid** rank token.

Consider the overall probability of the sequence from the first token up to each candidate option, shown in Figure 2b. For example, the probability of `matrix_rank` is roughly 60.21%. Eliminating invalid options, we can renormalize the probabilities of all remaining valid options. This process ensures that all valid options experience an equivalent proportional increase in their probabilities.

We can reorganize the probabilities, resulting in Figure 2c. The probability of `matrix` has decreased from 60.46% to 0.55%, which aligns more closely with our intuition that, after learning that the top candidate `matrix_rank` is invalid, the model should aim to generate the second candidate, which does not begin with `matrix`. This happens because we have performed a global probability adjustment after eliminating the invalid option `matrix_rank`, thereby allowing the probability of `matrix` to correctly decrease.

Now we want to **backtrack and change the initial choice** of `matrix`, so that we can follow the output intent of the model to not generate `matrix` and circumvent the limitation of constrained decoding. But now comes the question: given that we have already generated `matrix` with a probability of 60.46%, how to adjust this probability to 0.55%? One option is to directly resample. In practice, however, this would imply that we would resample the entire

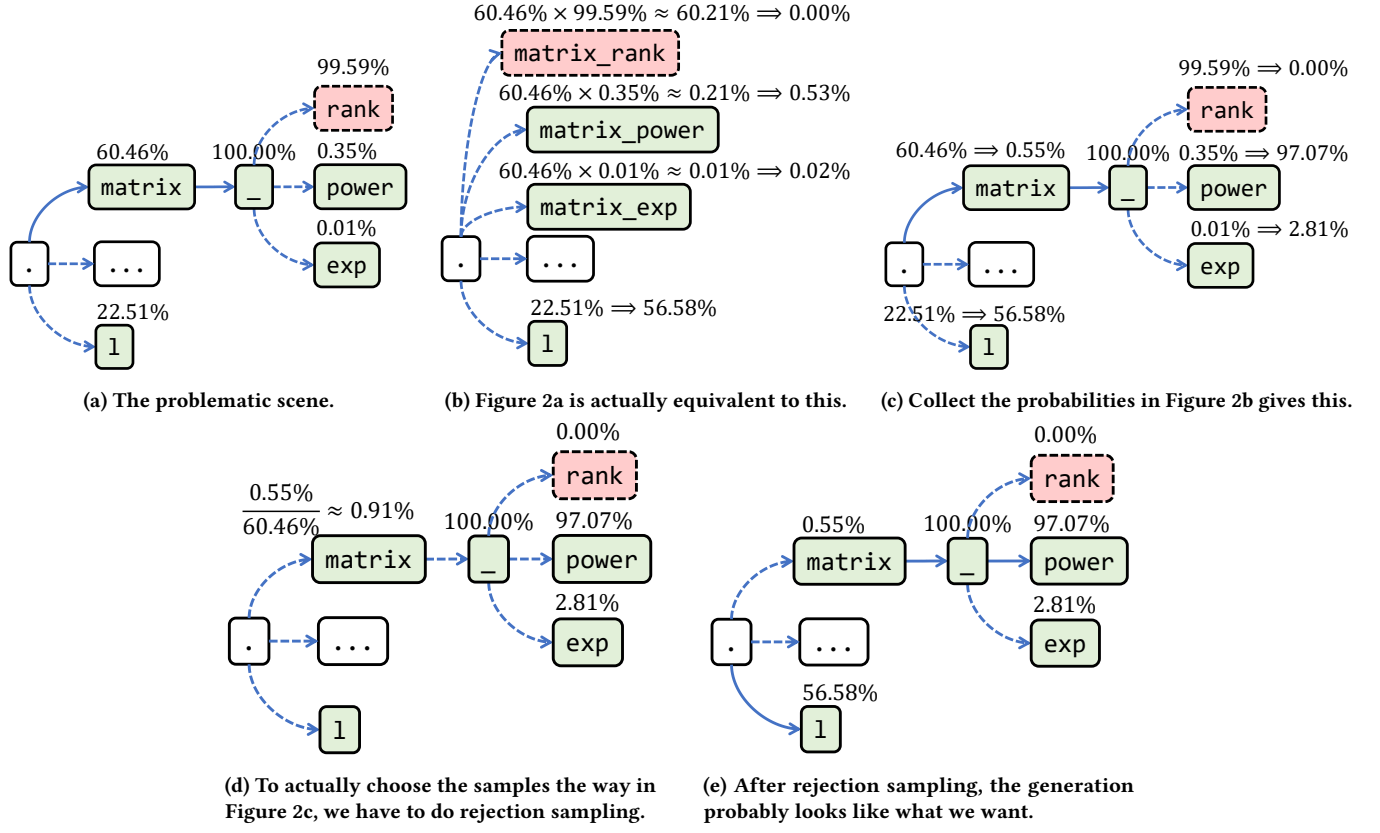


Figure 2: Motivating example: how backtracking works on the example in Figure 1.

sequence regardless of how minor the new error identified by the constrainer is, which would be wasteful of time and computational resources. We have an intuition that if the probability change is substantial, we are likely inclined to depart from this branch; otherwise, we probably wish to remain within the same branch.

To formalize this intuition, we propose the use of rejection sampling for **adaptive backtracking**, as shown in Figure 2d. We have generated `matrix` with a probability of 60.46%. We can perform a second sampling with an acceptance rate of roughly 0.91%. If the sampling succeeds, we can continue to use `matrix`; otherwise, we switch to another token. Thus, by combining the results of the two sampling processes, we achieve a sampling outcome where `matrix` is adopted with a probability of 0.55%.

After this resampling, one possible result is shown in Figure 2e, where the first token is changed to 1. This directs us towards the generation of the second candidate, `linalg.matrix_rank`.

3 Background

3.1 Goal

We have demonstrated, by example, that the output of constrained decoding can be far away from the model's intent, which can be shown by the large difference between the probability distribution of constrained decoding and that of the language model.

To keep the output aligned with the model's intent, the best option is to perfectly align with the probability distribution of the language model while keeping the constraints. In this subsection, we will formalize this goal.

Given a context $s_{<i}$, an autoregressive language model can produce a distribution over the next token $P(s_i | s_{<i})$. This allows us to define a distribution over all possible sequences, i.e., the joint probability distribution of the entire sequence,

$$P(s_0, \dots, s_n) = \prod_{i=0}^n P(s_i | s_{<i}).$$

A constrainer is a function defined on a sequence $c : s \mapsto \{0, 1\}$, where the result is 0 if all sequences that start with this sequence as their prefix is invalid, and 1 otherwise. We use $C(s) = \{t | c(s :: [t]) = 1\}$ to represent all valid tokens after the prefix s .

Our task is to define a distribution P_c over all valid sequences, such that the probability of generating any sequence is proportional to its probability under the language model. This can be formally expressed as follows,

$$P(s|c) \propto P(s)c(s), \forall s \text{ that is complete.}$$

Or equivalently, for any two valid complete sequences s and s' , we should have the following property,

$$\frac{P(s|c)}{P(s'|c)} = \frac{P(s)}{P(s')}. \quad (1)$$

3.2 Constrained decoding and its problem

Constrained decoding is an ad-hoc modification of rejection sampling, where illegal options are directly removed during the sampling process. Specifically, instead of repeatedly sampling and discarding invalid sequences, constrained decoding eliminates invalid options at each step, ensuring that only valid tokens are considered for generation.

Consider the generation probability of a valid sequence $s = (s_0, \dots, s_n)$ under constrained decoding. At each step of sampling s_i , the probability can be viewed as a form of rejection sampling, where the probability of sampling s_i given the prefix $s_{<i}$ is normalized over all valid continuations. Specifically, the probability of sampling s_i is given by the following equation,

$$P_{\text{con}}(s_i | s_{<i}) \propto P(s_i | s_{<i}) \mathbb{I}[s_i \in C(s_{<i})],$$

$$P_{\text{con}}(s_i | s_{<i}) = \frac{P(s_i | s_{<i}) \mathbb{I}[s_i \in C(s_{<i})]}{\sum_{t \in C(s_{<i})} P(t | s_{<i})}.$$

Let us compare the generation probabilities of two valid sequences s and s' .

$$\frac{P_{\text{con}}(s)}{P_{\text{con}}(s')} = \frac{P(s)}{P(s')} \frac{\prod_{i=0}^{n'} \sum_{t \in C(s'_{<i})} P(t | s'_{<i})}{\prod_{i=0}^n \sum_{t \in C(s_{<i})} P(t | s_{<i})}.$$

When compared to the equation 1, the second term in the product, which depends on the sums over $C(s_{<i})$ and $C(s'_{<i})$, cannot be canceled out. This shows that the generation probabilities of sequences under constrained decoding are distorted relative to their original language model probabilities.

4 AdapTrack

After defining our goal and necessary notations in section 3, we can now describe our algorithm AdapTrack. Overall, our algorithm keeps track of the validity of all known incomplete sequences and maintains a sample for each prefix based on the validity information. Our algorithm repeatedly samples an incomplete sequence and collects the validity information into the sample. We compare the validity information before and after this sampling and perform rejection sampling to correct the samples for each prefix. When the rejection happens, our algorithm will next time select a different token on this prefix. This is equivalent to **backtracking**, because we no longer use the same partially generation sequence.

Let us start from the beginning. We want to sample the distribution $P(s|c)$, which can be decomposed as a series of sampling from $P(s_i|c, s_{<i})$ as follows,

$$P(s|c) = \prod_i P(s_i|c, s_{<i}).$$

$P(s_i|c, s_{<i})$ can be decomposed by the Bayesian rule as follows,

$$P(s_i|c, s_{<i}) \propto P(s_i|s_{<i})P(c|s_{<i} :: [s_i]).$$

For every prefix x , we define our current estimation of $P(c|x)$ as $Q[x]$, where Q is an associative array. We call a prefix x is **new** if we have not calculated $P(\cdot|x)$; otherwise we call it **old**. If the prefix x is invalid, $Q[x]$ should be 0; otherwise, if x is old, which means we know possible valid sequences starting from x , we simply aggregate the information; otherwise, we just assume that its children are

always valid and use 1 as an estimation.

$$Q[x] := \begin{cases} c(x) & \text{if } c(x) = 0 \text{ or } x \text{ is new;} \\ \sum_u P(u|x)Q[x :: [u]] & \text{if } c(x) = 1 \text{ and } x \text{ is old.} \end{cases} \quad (2)$$

We use $P_Q(\cdot)$ to denote the distribution of $P(\cdot|c)$ estimated with Q ,

$$P_Q(s_i|s_{<i}) \propto P(s_i|s_{<i})Q[s_{<i} :: [s_i]]. \quad (3)$$

For every prefix x we have encountered during decoding, we maintain a next token $N[x]$ sampled from $P_Q(\cdot|x)$, where N is an associative array. We can concatenate all the next token samples to get a sample from $P_Q(\cdot)$, as shown in algorithm 1.

Algorithm 1 Collect a sequence from next tokens

```

1: procedure COLLECTSEQUENCE( $N$ )
2:    $s \leftarrow \epsilon$ 
3:   while CONTAINS( $N, s$ ) do
4:      $s \leftarrow s :: [N[s]]$ 
5:   end while
6:   return  $s$ 
7: end procedure

```

Our decoding process first samples a prefix s from $P_Q(\cdot)$ as indicated by N . If it is a complete sequence, we can just return it. Otherwise, we call the constrainer for every possible next token and record their information in Q accordingly. We sample $N[s]$ from all the valid next tokens by their probability from the language model (the same distribution as $P_Q(\cdot|s)$). We update $Q[x]$ for all prefixes x of the sequence s and decide **whether we want to backtrack** at this prefix by updating the prefix's next token $N[x]$.

The estimated validity $Q[x :: [N[x]]]$ of the current next token $N[x]$ decreases during the update because we always overestimate the validity. This reduces the probability of drawing $N[x]$ from $P_Q(\cdot|x)$, as shown in formula 3. We decide whether we continue to use $N[x]$ as the sample from the updated $P_Q(\cdot|x)$ by rejection sampling. Denote Q before updating as Q , and that after updating as Q' . We sample a Bernoulli distribution with the probability of $P_{Q'}(N[x]|x)/P_Q(N[x]|x)$ to keep the current next token $N[x]$ unchanged, and otherwise change to other tokens¹. If we change to another next token, which means **backtracking occurs**, we sample a new next token from all **other** tokens with probability $P_Q(\cdot|c, x)$, but we explicitly exclude the old $n := N[x]$.

$$P'_Q(n'|x, n) \propto P_Q(n'|x) \mathbb{I}[n' \neq n]. \quad (4)$$

Collectively, our procedure can be summarized in algorithm 2.

5 The proof of AdapTrack

AdapTrack focuses on not distorting the model distribution under constraint and, therefore, not distorting its output intent. We now give a proof of this statement.

THEOREM 5.1. *Algorithm 2 only returns a valid complete sequence, and the probability of generating different sequences that have been explored during decoding satisfies equation 1.*

¹This probability is different from $Q'[x :: [N[x]]]/Q[x :: [N[x]]]$, because the denominator in $P_Q(\cdot|x)$ is also changed.

Algorithm 2 AdapTrack

```

1: procedure BACKTRACKSAMPLE( $P_{LM}, c$ )
2:    $Q \leftarrow \emptyset$  ▷ Estimate of  $P(c|s)$  for all known  $s$ 
3:    $N \leftarrow \emptyset$  ▷ Sample of  $P_Q(\cdot|s)$  for all known  $s$ 
4:   while true do
5:      $s \leftarrow \text{COLLECTSEQUENCE}(N)$ 
6:     if  $s_{-1}$  is complete then
7:       return  $s$ 
8:     end if
9:      $N[s] \sim P_Q(\cdot|s)$ 
10:    Update  $Q$  by formula 2, denote the updated  $Q$  as  $Q'$ 
    ▷ Update from the longest sequence to the shortest
11:    for each prefix  $x$  of  $s$  excluding  $s$  itself do
12:       $n \leftarrow N[x]$  ▷ The old  $N[x]$ 
13:       $p \leftarrow P_Q(n|x)$ 
14:       $p' \leftarrow P_{Q'}(n|x)$ 
15:       $b \sim \text{Bernoulli}(p'/p)$  ▷ Rejection sample
16:      if  $b = 0$  then ▷ Backtrack
17:         $N[x] \sim P_{Q'}(\cdot|x, n)$  as formula 4
18:      end if
19:    end for
20:  end while
21: end procedure

```

PROOF. The only **return** statement is step 7, so the returned sequence is always complete. Also, the returned sequence is collected from N , where $N[x] \sim P_Q(\cdot|x) \propto P(\cdot|x)Q[x :: [\cdot]]$ always guarantees that $N[x]$ is a valid subsequent token for any prefix x , so the algorithm always returns valid well-ended sequences.

Collecting one sample sequence from N is equivalent to sampling a result from $P_Q(\cdot)$ because we always maintain this property in the algorithm by rejection sampling, as shown in theorem 5.2.

For any new sequence $s = (s_0, \dots, s_n)$,

$$\begin{aligned}
 P_Q(s) &= \prod_{i=0}^n P_Q(s_i | s_{<i}) = \prod_{i=0}^n \frac{P(s_i | s_{<i}) Q[s_{<i} :: [s_i]]}{\sum_u P(u | s_{<i}) Q[s_{<i} :: [u]]}, \\
 &= \prod_{i=0}^n \frac{P(s_i | s_{<i}) Q[s_{<i}]}{Q[s_{<i}]} = \frac{Q[s]}{Q[\varepsilon]} P(s) = \frac{1}{Q[\varepsilon]} P(s). \quad \square
 \end{aligned}$$

THEOREM 5.2. *The update process of N in steps 11 to 19 in algorithm 2 makes $N[x]$ follows distribution $P_Q(\cdot|x)$ for prefix x .*

PROOF. We give a proof by induction on the update of $N[x]$.

Base case. In step 9, $N[x]$ is drawn from $P_Q(\cdot|x)$.

Induction step. When the distribution $Q[x]$ is updated in step 10, denote the original $N[x]$ by n .

By rejection sampling with an acceptance rate of $P_{Q'}(n|x)/P_Q(n|x) \leq 1$, the total probability of generating n as a sample after the prefix x is as follows.

$$P_Q(n|x) \times \frac{P_{Q'}(n|x)}{P_Q(n|x)} = P_{Q'}(n|x).$$

The probability of generating another token $n' \neq n$ as a sample after the prefix x is as follows.

$$\begin{aligned}
 &P_Q(n'|x) + P_Q(n|x) \times \left(1 - \frac{P_{Q'}(n|x)}{P_Q(n|x)}\right) \times \frac{P_Q(n'|x)}{1 - P_Q(n|x)}, \\
 &= (1 - P_{Q'}(n|x)) \times \frac{P_Q(n'|x)}{1 - P_Q(n|x)}, \\
 &= (1 - P_{Q'}(n|x)) \times \frac{P(n'|x)Q[x :: [n']]}{\sum_{u \neq n} P(u|x)Q[x :: [u]]}, \\
 &= (1 - P_{Q'}(n|x)) \times \frac{P(n'|x)Q'[x :: [n']]}{\sum_{u \neq n} P_{LM}(u|x)Q'[x :: [u]]}, \\
 &= (1 - P_{Q'}(n|x)) \times \frac{P_{Q'}(n'|x)}{1 - P_{Q'}(n|x)} = P_{Q'}(n'|x). \quad \square
 \end{aligned}$$

6 Evaluation

We have proven AdapTrack aligns with the language model in section 5. In this section, we evaluate AdapTrack against other decoding methods to answer the following research questions:

RQ1. How does AdapTrack perform in predicting correct and valid APIs?

RQ2. How does AdapTrack perform in predicting correct and valid APIs in real-world scenarios?

RQ3. How does AdapTrack perform in general code generation?

RQ4. How does AdapTrack align with the oracle distribution of language models under constraint?

6.1 RQ1: Performance of AdapTrack in predicting correct and valid APIs

As the library evolves, it often happens that an old API is transitioned into another new API. If the user has updated their library, only the new APIs should be used; if not, only the old APIs should be used. In RQ1, we aim to simulate this scenario. We ask the model to use the same API with the old and the new versions of the same library, respectively. The APIs used by the model should not only **have the correct semantics**, but also **be valid in the corresponding library version**. With the new library, the model should adopt the new API; with the old one, it should stick to the old API.

```

import tensorflow as tf

def mean_pairwise_squared_error(*args, **kwargs):
    """
    wrapper for losses.mean_pairwise_squared_error
    """
    return tf|

```

Figure 3: An example code in the TFv1 dataset.

Dataset. We construct a dataset, TFv1, based on the update from TensorFlow v1 to v2. We prompt the model to generate an API unique to v1 in a context where it cannot infer the library version and then evaluate its ability to correctly generate the API

Table 1: Exact match on TFv1 in different TensorFlow versions.

Method	TensorFlow v1 setting					TensorFlow v2 setting				
	EM@1	EM@3	EM@5	EM@10	EM@20	EM@1	EM@3	EM@5	EM@10	EM@20
<i>Qwen2.5 Coder 7B</i>										
Unconstrained decoding	61.58%	73.07%	81.51%	88.85%	92.84%	7.16%	19.04%	26.17%	37.77%	50.60%
Constrained decoding	66.11%	76.35%	83.50%	90.10%	94.03%	18.38%	31.84%	39.63%	50.34%	60.86%
AdapTrack (Ours)	80.19%	87.36%	91.86%	95.96%	98.33%	54.89%	62.51%	68.87%	76.52%	82.82%
<i>DeepSeek Coder Base 6.7B</i>										
Unconstrained decoding	62.53%	68.63%	78.50%	88.43%	94.27%	5.97%	16.91%	24.12%	36.49%	50.60%
Constrained decoding	65.63%	73.34%	82.03%	90.26%	94.75%	10.98%	25.31%	33.32%	45.90%	59.43%
AdapTrack (Ours)	83.29%	86.98%	92.47%	96.48%	98.09%	50.60%	56.62%	64.76%	74.57%	82.82%
<i>StarCoder2 7B</i>										
Unconstrained decoding	72.79%	76.37%	85.12%	92.30%	96.18%	1.67%	12.66%	18.62%	29.76%	44.15%
Constrained decoding	73.99%	79.99%	87.36%	93.23%	96.42%	10.26%	21.48%	28.83%	40.48%	53.70%
AdapTrack (Ours)	83.53%	88.78%	92.97%	96.36%	98.33%	45.35%	50.90%	59.70%	69.75%	77.57%
<i>CodeLlama Python 7B</i>										
Unconstrained decoding	59.90%	70.63%	78.86%	86.61%	91.41%	11.93%	25.20%	32.52%	43.46%	55.37%
Constrained decoding	66.35%	76.36%	83.47%	89.78%	93.56%	21.24%	38.35%	46.12%	56.96%	67.54%
AdapTrack (Ours)	83.29%	89.37%	92.82%	95.70%	97.14%	60.86%	68.20%	73.37%	79.56%	85.20%

in both v1 and v2 settings. The version information is given to the constrainer, so the constrainer is aware of all the valid APIs in the setting. To guide the model toward using a specific API, we explicitly annotate the function as a wrapper, encouraging the model to directly forward calls to this API. An example is illustrated in Figure 3. For this example, the correct answer is `.losses.mean_pairwise_squared_error` and `.compat.v1.losses.mean_pairwise_squared_error` for TensorFlow v1 and v2 settings, respectively.

We collect all legal APIs under TensorFlow 2.16 from the official TensorFlow website. From them, we filter all APIs starting with `tf.compat.v1`. We exclude APIs that have counterparts in v2 based on the last part of the API name. To avoid interference from classes and methods, we retain only APIs where all parts of its name start with a lowercase letter. To avoid interference from subpackages, we remove all APIs that can be a prefix of another API. Ultimately, we obtained 419 standalone TensorFlow v1-specific APIs.

Metrics. We use exact match (EM) metrics. We report EM@ k [9],

$$\text{EM@}k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right], \quad (5)$$

where n is the sample size, c is how many samples exactly match the oracle in n samples, and k is in 1, 3, 5, 10, 20. EM@ k reflects the expected value of achieving an exact match within k attempts.

For each method on each problem in each setting, we sample one result by greedy decoding and then sample 20 results randomly. For EM@1, we use the greedy result, and the sample size is 1. For EM@ k where $k > 1$, we calculate the metrics from the 20 samples. Because unconstrained decoding does not depend on the version information, we reuse the same sampling results and only change the oracle between the two settings.

Baselines. We compare with two baselines, unconstrained decoding and constrained decoding. Unconstrained decoding is decoding without a constrainer. At each time step, the next token is sampled

from the model distribution without filtering. Constrained decoding is the decoding method we describe in section 3. At each time step, the probabilities of invalid tokens are set to 0 in the model distribution before sampling.

Implementation. For the TensorFlow v2 setting, we use all collected TensorFlow APIs as valid APIs in the constrainer. For TensorFlow v1 setting, we use all the v1 APIs as valid APIs in the constrainer, but we remove their additional prefix `compat.v1`, to keep the same syntax as in TensorFlow v1. We implement the AdapTrack algorithm with PyTorch [5] and Transformers [36] library. We conduct experiments on a 64-core Intel Xeon machine with 8 NVIDIA RTX A6000 GPUs.

Models. We conduct experiments on Qwen2.5 Coder 7B [16], DeepSeek Coder Base 6.7B [13], StarCoder2 7B [25] and CodeLlama Python 7B [31]. We only use base models in this experiment, because the task of code completion aligns better with the training target of the base models than that of the instruct models.

Results. As shown in Table 1, **for both TensorFlow v1 and v2 settings, AdapTrack performs significantly better than constrained decoding, with an improvement up to 360.87%.**

In the TensorFlow v1 setting, Qwen2.5 Coder 7B can already achieve an EM@1 of 61.58%. With a constrainer, the result can be improved to 66.11%, so the model may still generate some API calls invalid in TensorFlow v1 and eliminated by the constrainer. But using AdapTrack, the result can be further improved to 80.19%, indicating that, in roughly 14% cases, the model considers the oracle as its best candidate under constraint but fails to generate it, because constrained decoding does not allow the model to modify already generated tokens. Since AdapTrack does allow the model to modify them to better express its output intent, AdapTrack performs significantly better than simple constrained decoding.

It is more dramatic in TensorFlow v2. Without a constrainer, Qwen2.5 Coder 7B can only achieve an EM@1 of 7.16%. This result

Table 2: Exact match on Tfv1Real.

Method	Exact Match @ k ($\uparrow$)				
	EM@1	EM@3	EM@5	EM@10	EM@20
<i>Qwen2.5C 7B</i>					
Unconstrained	21.70%	29.45%	35.28%	43.88%	53.00%
Constrained	35.60%	41.86%	47.64%	55.29%	62.30%
AdapTrack	49.30%	53.87%	59.28%	65.84%	71.70%
<i>DSC Base 6.7B</i>					
Unconstrained	18.90%	25.62%	30.93%	38.66%	47.10%
Constrained	31.10%	38.45%	44.09%	51.23%	57.70%
AdapTrack	41.90%	49.45%	55.26%	62.25%	68.30%
<i>SC2 7B</i>					
Unconstrained	15.90%	21.91%	27.33%	35.59%	44.70%
Constrained	29.80%	35.55%	41.69%	49.59%	56.60%
AdapTrack	41.40%	47.60%	53.85%	61.62%	68.10%
<i>CL Python 7B</i>					
Unconstrained	19.60%	26.88%	32.26%	40.48%	49.40%
Constrained	34.40%	40.77%	46.99%	55.10%	62.30%
AdapTrack	46.80%	52.61%	58.75%	66.41%	73.00%

should be considered together with EM@1 in the TensorFlow v1 environment. For 61.58% cases, the model wants to generate v1-compliant APIs as the best candidate, and then it cannot generate v2-compliant APIs by unconstrained decoding. If we add a constrainer, this result can be improved to 18.38%, because APIs that are invalid in TensorFlow v2 cannot be generated now.

Now consider the results of AdapTrack. With the help of the constrainer to remove invalid APIs, the model can now freely generate the best remaining option in its candidates, resulting in an EM@1 of 54.89%, which is an improvement of **198.70%** over the 18.38% EM@1 of constrained decoding. This is even better than EM@10 of constrained decoding, further showing the importance of not distorting the language model’s output intent.

Similar analysis can be applied to the results of other models, while the biggest improvement, **360.87%** compared to constrained decoding, happens at EM@1 of DeepSeek Coder Base 6.7B in TensorFlow v2 environment, from 10.98% to 50.60%.

6.2 RQ2: Performance of AdapTrack in real-world API completion

We have found that AdapTrack is extremely effective on `tf.compat.v1` APIs in RQ1, but the results come from a synthetic dataset, Tfv1. We want to know whether this phenomenon also occurs in real-world code completion.

Dataset. We collect real usage of such APIs from GitHub. For every v1 API in Tfv1, we search for `tf.compat.v1.API()`, and collect 46,785 Python files. We deduplicate these files and split each file on the first usage of `tf.compat.v1` in code. We then remove all files that contain `tf.compat.v1` in the prefix. If the suffix does not start with an API call, the file is also removed. After these cleaning steps, we now obtain 14,237 files. We use the prefix as the completion prefix and the starting API call in the suffix as the oracle.

We then calculate the length distribution of the prefixes. We use the tokenizer of CodeLlama 7B Python because it has a relatively

small vocabulary of 32,000 tokens, which makes it easier to overflow the model’s max position after tokenization. The average length of the prefixes is 1,624.78 tokens, and 90% of the prefixes have less than or equal to 3,463.0 tokens. To make the dataset friendlier to models with smaller context windows, we remove files whose prefix is longer than $4,096 - 512 = 3,584$ tokens. This gives us 12,883 files. We then randomly pick 1,000 files as the Tfv1Real dataset.

Results. As shown in Table 2, **AdapTrack performs significantly better than constrained decoding for all models, with an improvement up to 38.93%.**

Take Qwen2.5 Coder 7B as an example. Qwen2.5 Coder 7B can already correctly generate 21.70%. With a constrainer to eliminate invalid options, the EM@1 can be further improved to 35.60%, indicating that the model is generating many invalid APIs when not constrained. With AdapTrack, this result can be further improved to 49.30%, an improvement of 38.48% compared to the result of constrained decoding, indicating that in constrained decoding, the model is forced to generate many suboptimal options that are misaligned with the model’s output intent. The improvement is not as significant as in the Tfv1 dataset. Nevertheless, 49.30% is still a large improvement.

Similar analysis can be applied to the results of other models, while the largest improvement, **38.93%** compared to constrained decoding, happens in the results of StarCoder2 7B at EM@1, from 29.80% to 41.40%.

6.3 RQ3: Performance of AdapTrack in general code generation

In RQ1 and RQ2, we conduct experiments on two API completion datasets, Tfv1 and Tfv1Real. Now we want to know whether AdapTrack is effective in general code generation.

Dataset. We adopt the widely used HumanEval [9] and MBPP [6] benchmarks. We use a type constrainer [26] to ensure the generated programs are well-typed. Because the constrainer is only available for programs written in TypeScript, we use the TypeScript-translated versions from the MultiPL-E dataset [8].

Metrics. We report `pass@k` [9], which is given by replaces c in equation 5 with how many samples can pass the test.

Sampling strategy. It is too time-consuming to check the validity of the whole vocabulary after every prefix. We introduce a variant of top- p (nucleus) sampling [15] to reduce the tokens to check. We check the vocabulary from the most possible token to the least possible one. After calculating the validity of each token, we calculate the total probability of known valid tokens, and its ratio to the total probability of possibly valid tokens (i.e. known valid tokens + unchecked tokens). Once this ratio exceeds p , all remaining unchecked tokens are considered invalid. In unconstrained decoding, this sampling strategy naturally becomes the classic top- p sampling, because every token is considered valid.

Implementation. We modify the constrainer to fix several bugs and speed up the calculation. We open-source the modified package together with our source code.

We restart generation when the constrainer cannot determine validity of one token in 60 seconds. If greedy sampling is restarted

Table 3: Pass@ k on general-purpose code generation benchmarks.

Method	HumanEval					MBPP				
	pass@1	pass@3	pass@5	pass@10	pass@20	pass@1	pass@3	pass@5	pass@10	pass@20
<i>Qwen2.5 Coder 7B</i>										
Unconstrained decoding	65.41%	63.05%	73.66%	84.36%	90.57%	66.92%	72.28%	79.10%	85.04%	88.72%
Constrained decoding	64.15%	70.66%	79.00%	86.08%	89.31%	66.67%	75.13%	80.60%	85.56%	88.46%
AdapTrack (Ours)	69.18%	72.22%	79.98%	86.65%	92.45%	70.77%	76.26%	81.55%	86.59%	89.74%
<i>DeepSeek Coder Base 6.7B</i>										
Unconstrained decoding	44.65%	46.52%	56.26%	68.23%	77.36%	36.41%	65.62%	73.77%	81.17%	85.38%
Constrained decoding	47.17%	51.88%	60.36%	70.00%	78.62%	60.00%	67.87%	74.26%	80.24%	84.10%
AdapTrack (Ours)	49.69%	55.43%	63.84%	73.16%	80.50%	63.85%	70.29%	76.54%	82.26%	85.90%
<i>StarCoder2 7B</i>										
Unconstrained decoding	37.74%	28.00%	35.79%	46.85%	57.86%	56.67%	45.94%	55.72%	66.36%	74.36%
Constrained decoding	37.11%	37.67%	46.35%	58.18%	68.55%	52.05%	57.43%	65.09%	73.17%	78.72%
AdapTrack (Ours)	37.74%	40.72%	49.62%	60.75%	69.18%	54.36%	58.60%	66.17%	74.33%	80.51%
<i>CodeLlama 7B</i>										
Unconstrained decoding	33.96%	32.15%	39.50%	49.89%	61.64%	48.72%	45.68%	54.57%	64.42%	71.54%
Constrained decoding	34.59%	35.19%	42.62%	53.43%	65.41%	45.90%	52.25%	60.19%	68.89%	75.64%
AdapTrack (Ours)	36.48%	35.89%	43.41%	53.89%	64.15%	47.69%	53.26%	61.25%	70.34%	77.18%

20 times for the same problem, since it always returns the same answer for the same problem, the sample is considered empty.

Results. As shown in Table 3, **AdapTrack almost always outperforms constrained decoding, with an improvement of pass@1 up to 7.84% on HumanEval and 6.42% on MBPP, compared to constrained decoding**, indicating that **distortion also widely happens to constrained decoding in general code generation**. Fixing it with AdapTrack improves the performance. The only outlier is pass@20 of CodeLlama 7B on HumanEval.

When compared to unconstrained decoding, AdapTrack sometimes performs worse (pass@1 of StarCoder2 7B and CodeLlama 7B on MBPP). This is due to **the constrainer’s limitation**. Note that pass@1 is calculated from the greedy samples. If the greedy samples of unconstrained decoding can pass the test, it should pass the constrainer and be greedily sampled by constrained decoding. However, in these cases, unconstrained decoding outperforms constrained one. Nevertheless, AdapTrack can still improve the performance of constrained decoding by fixing the distribution distortion.

6.4 RQ4: Alignment between AdapTrack and the oracle distribution

We have conducted experiments on both API completion and code generation. Now we want to verify the statement that the distribution produced by AdapTrack aligns better with the oracle distribution, as an explanation of the previous results. Some previous work [27] also focuses on fixing the language model’s probability in constrained decoding, so we follow their work and duplicate their experiments on AdapTrack in RQ4.

Datasets. Following previous work [27], we use 4 datasets in domain-specific languages.

Metrics. Following previous work [27], we generate 2000 samples for each problem in the SLIA, INV-BV and CP datasets, and

100 samples for the binary dataset. We use as metrics the Kullback-Leibler (KL) divergence between the distribution formed by the samples and the language model distribution. **Lower the KL divergence, the closer the two distributions.** For any distribution Q that only contains constrained samples, the KL divergence between Q and P_{LM} reflects the KL divergence between Q and $P(\cdot|c)$.

$$\begin{aligned}
 D_{KL}(Q||P(\cdot|c)) &= \sum_s Q(s) \log \frac{Q(s)}{P(s|c)} = \sum_s Q(s) \log \frac{Q(s)}{P_{LM}(s)Z}, \\
 &= \sum_s Q(s) \log \frac{Q(s)}{P_{LM}(s)} - \log Z = D_{KL}(Q||P_{LM}) - \log Z,
 \end{aligned}$$

where Z is the normalizing constant in $P(\cdot|c)$.

It should be noted that these metrics are only meaningful when compared with results on the same problem because an identical unknown constant $\log Z$ has been added to all of them.

For each question, we calculate the average KL divergence between all the samples and the language model. We also present the average KL divergence of the four datasets, weighted by the number of problems in each dataset.

To see how each method converges to the oracle distribution, we also calculate the KL divergence between a sliding window of n samples and the language model, where n is one-fourth of the sampling size for each problem.

Baselines. We compare with two baselines, constrained decoding (CD) and Adaptive Sampling with Approximate Expected Futures (ASAp) [27]. ASAp can be regarded as a variant of AdapTrack, where after sampling a sequence from P_Q , instead of backtracking and adjusting all the previous sampling results, it just continues to use the current partially generated prefix. It uses Q only for the later samples on the same problem. To match its behavior, we also share the estimation data Q between different generations; only the sample data N is updated for a new sample.

Implementation. We reuse ASAp code to replicate their settings and the same transformers-CFG [11] library for the constrainer.

Models. Following previous work [27], we use the Mistral-7B [17] model for all decoding algorithms.

Table 4: Comparison of KL divergence of different methods on DSL datasets. CD means constrained decoding.

Method	KL-divergence ($\downarrow$)				
	SLIA	INV-BV	CP	binary	average
CD	11.37	7.13	19.86	23.05	11.46
ASAp	12.05	7.76	19.85	19.87	11.90
AdapTrack	9.31	7.04	16.86	19.53	9.97

Results. As shown in Table 4, **in all four datasets, AdapTrack performs significantly better than both constrained decoding and ASAp.** In both the SLIA and CP datasets, constrained decoding and ASAp perform roughly the same, while AdapTrack has a large advantage. In the INV-BV dataset, all three methods perform roughly the same. This might indicate that the portion of illegal tokens on different tokens in this dataset is roughly the same, so constrained decoding can perform roughly the same as AdapTrack. The binary dataset is the only dataset where ASAp performs much better than constrained decoding, while AdapTrack still has some advantage over ASAp.

Figure 4 shows how the KL divergence evolves in repeated sampling. The point with the abscissa x in the figure represents the KL divergence between the distribution formed by the x th to the $(n+x)$ th samples and the model distribution.

With more and more information stored in Q , decoding results of AdapTrack become better and better. On the SLIA, CP and binary dataset, AdapTrack has a low KL divergence from the start and continues to be low for the entire generation, indicating that AdapTrack has already found a good distribution from the first one-fourth samples. On the INV-BV dataset, the KL divergence of AdapTrack declines greatly, indicating that AdapTrack is still learning new things in repeated sampling.

7 Discussion

7.1 Robustness analysis

Performance of decoding methods often depends on several factors, such as sampling temperature and model size. Therefore, we conduct additional experiments, and show that AdapTrack consistently outperforms constrained decoding under different temperatures and model sizes. See supplementary material for more details.

7.2 Efficiency and scalability of AdapTrack

One might concern the efficiency and scalability of AdapTrack from two perspectives: (1) updating Q and N from step 10 to step 19 in algorithm 2 can be inefficient; (2) AdapTrack may need too much time before generating a complete sequence.

7.2.1 Parallel update. The update of Q and N can be parallelized on GPU, rather than performed token by token. If $Q[x]$ is updated

Table 5: Numbers of calls to language model in RQ1.

Method	calls to LM ($\downarrow$)	
	TFv1 (v1 settings) avg (min, max)	TFv1 (v2 settings) avg (min, max)
<i>Qwen2.5 Coder 7B</i>		
Unconstrained decoding	5.85 (4.12, 8.47)	5.85 (4.12, 8.47)
Constrained decoding	4.64 (2.75, 6.07)	5.04 (2.91, 7.82)
AdapTrack (Ours)	5.56 (4.12, 7.42)	7.94 (4.41, 11.73)
<i>DeepSeek Coder Base 6.7B</i>		
Unconstrained decoding	10.36 (6.32, 15.59)	10.36 (6.32, 15.59)
Constrained decoding	8.27 (4.93, 11.43)	8.82 (4.74, 14.40)
AdapTrack (Ours)	9.85 (6.24, 11.37)	13.90 (6.70, 22.87)
<i>StarCoder2 7B</i>		
Unconstrained decoding	9.11 (5.75, 13.65)	9.11 (5.75, 13.65)
Constrained decoding	7.62 (4.47, 10.03)	7.70 (4.33, 12.50)
AdapTrack (Ours)	8.83 (5.93, 12.60)	12.08 (5.47, 20.32)
<i>CodeLlama Python 7B</i>		
Unconstrained decoding	10.68 (7.56, 12.90)	10.68 (7.56, 12.90)
Constrained decoding	8.41 (5.38, 10.75)	9.73 (5.88, 14.02)
AdapTrack (Ours)	10.03 (7.36, 13.45)	14.29 (8.81, 20.25)

from 1 to q , where x becomes an old prefix during the update, if $x = \text{CONCAT}(s, x')$, following formula can be used to update $Q[s]$,

$$Q[s] \leftarrow Q[s] - P_{\text{LM}}(x'|s)(1 - q),$$

where $P_{\text{LM}}(x'|s)$ can be computed in parallel for all prefix s of x . It is then natural to compute the updated $P_Q(\cdot|s)$ in parallel, and perform the rejection sampling needed. It should be noted that the rejection sampling and the sampling after rejection can also be fused into one sampling, further reducing branching in the calculation.

7.2.2 Number of LM calls. The most computation-intensive part of the decoding process is the invocation of the language model. We count the number of calls to the language model per sample in the RQ1 experiments, as shown in Table 5. In the TensorFlow v1 setting, AdapTrack invokes the language model on average 20% more frequently than constrained decoding, whereas, in the TensorFlow v2 setting, this number rises to an average of 60% more calls than constrained decoding. Similar numbers also apply to the averaged maximum number of calls to LM. Considering that EM@1 of AdapTrack is often comparable to EM@5 of constrained decoding, this level of increase in the number of invocations can be justified.

If theoretical completeness is sacrificed, we can take additional constraints on AdapTrack to avoid extremely long backtracking distances, e.g., by setting a lower bound of the backtracking probability or an upper limit of the backtracking distance. We conduct additional experiments with limited backtracking distances in RQ1 experiments, and show that a short backtracking distance is enough in most cases. See the supplementary material for more details.

7.3 Threats to validity

7.3.1 Dataset selection. The first threat lies in the selection of dataset: Does this problem of constrained decoding happen only in deprecated APIs or only in API completion? We mitigate this

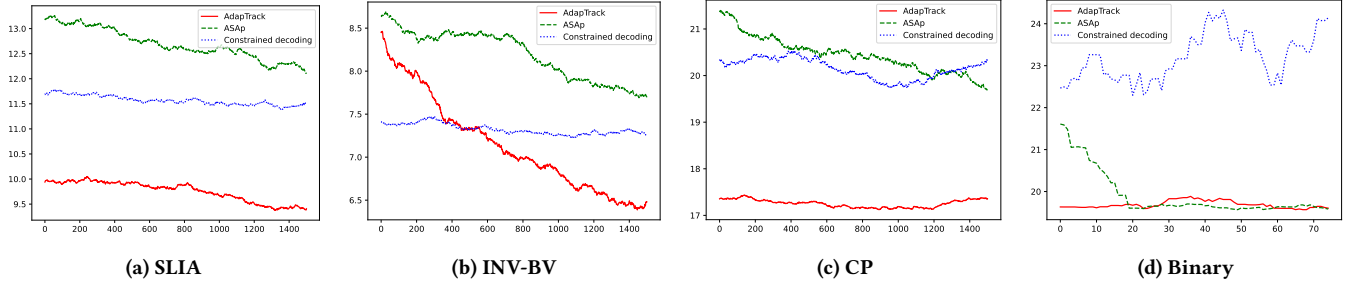


Figure 4: The change of KL divergence ($\downarrow$) between every 500 samples and the oracle distribution on different datasets.

threat by experiments in RQ1 where we ask the model to generate a TensorFlow v1 API in a TensorFlow v1 setting, where these APIs are not considered deprecated, which can be fixed by AdapTrack to provide significant improvement. Experiments on general code generation benchmarks in RQ3 show that such distortion also happens in general code generation, and AdapTrack can provide performance improvement by fixing such distortion.

7.3.2 Model selection. The second threat lies in the selection of the models: Is this issue due to the limitation in model’s capability? We mitigate this threat by using multiple models in experiments, and using differently-sized models in robustness analysis. Although they have different capabilities, AdapTrack shows a consistent improvement. As long as greedy constrained decoding is employed, this inherent problem will persist to some extent.

8 Related work

8.1 API-aware code generation

APIs evolve over time, and code LLMs need to handle that. CERT [38] introduces a sketcher to predict the sketch of API usage, and then uses a generator to use the API. Toolcoder [39] introduces an API search tool to help the model search for wanted APIs. Wang et al. [34] conduct the first evaluation study on the usage of deprecated APIs in LLM-based code completion. VersiCode [37] dataset focuses on version-specific code completion and version-aware code migration. CodeUpdateArena [24] dataset evaluates LLM’s ability to handle API update with synthetic API update.

8.2 Constrained decoding in code generation

Constrained decoding is a greedy method to ensure the generated sequence satisfy certain constraints. Picard [32] and Synchromesh [29] introduce incremental parser into code generation to help the model reject inadmissible tokens at each decoding step. Later grammar-based constrainers [7, 10, 21, 28] focus on the acceleration of constraint calculation. Monitor-guided decoding [3] and Repilot [35] introduce constrained decoding in general-purpose programming languages, by communicating with static analysis tools in IDEs, e.g., language servers in Visual Studio Code, to collect information about the whole repository to help API generation. Mündler et al. [26] introduce type system into constrained decoding, ensuring the generated program to be type-safe. CoSEFA [14] uses a security-aware model to ensure the generated code is secure.

8.3 Fixing greedy constrained decoding

Naïve greedy constrained decoding is known to be problematic, and there are some work trying to fix that. GeDi [22] introduces another trained model to model the expected validity of each prefix, but training another model is a large overhead. ASAp [27] samples multiple sequences from the language model and use the validity information from previous samples to direct future sampling. This can be regarded as a variant of AdapTrack without backtracking. Gen-C [4] models the constraint with constraint circuits and resamples using the pseudolikelihood in the neighborhood of a sentence, but constraint circuits are only constructible for regular expressions and, therefore, unsuitable for code generation.

8.4 Non-linear code generation techniques

Non-linear code generation techniques is not unheard of. PG-TD [40] uses Monte-Carlo tree search, using public test cases to determine the quality of generated code. Rocode [19] detects errors after generating every statement by static analysis tools and public test cases, and rollbacks to the statement of the error report or earlier statements with high uncertainty. However, they are not designed to address distribution distortion and consequently still produce distorted distributions, while AdapTrack specifically targets this issue and provides provably undistorted results.

9 Conclusion and future work

In this work, we propose a new decoding method, AdapTrack, to fix the distribution distortion problem in constrained decoding by introducing backtracking that is adaptive to the invalidated probability mass into the generation process. We collect the synthetic TFv1 dataset and the real-world TFv1Real dataset to verify the effectiveness of AdapTrack. Experiments on general code generation benchmarks verifies the generalizability of AdapTrack. We give theoretical proof that AdapTrack does follow the output intent of the model and conduct experiments on DSL problems to verify that.

In the future, we will explore this phenomenon on more APIs, and explore how to help the model align better with the constraints.

Acknowledgments

This research is supported by the National Key R&D Program under Grant No. 2023YFB4503801, the National Natural Science Foundation of China under Grant No. 62192733, 62192730, 62192731, and the Major Program (JD) of Hubei Province (No.2023BAA024).

References

- [1] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/> Software available from tensorflow.org.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu Lahiri, and Sriram Rajamani. 2023. Monitor-guided decoding of code lms with static analysis of repository context. *Advances in Neural Information Processing Systems* 36 (2023), 32270–32298.
- [4] Kareem Ahmed, Kai-Wei Chang, and Guy Van den Broeck. 2025. Controllable Generation via Locally Constrained Resampling. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=8g4XgC8HPF>
- [5] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [7] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation. In *Forty-first International Conference on Machine Learning*. <https://openreview.net/forum?id=pXaEYzrFae>
- [8] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2023. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3675–3691.
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [10] Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems* 7 (2024).
- [11] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 10932–10952. doi:10.18653/v1/2023.emnlp-main.674
- [12] GitHub. 2023. GitHub Copilot. <https://github.com/features/copilot>.
- [13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [14] Xuan He, Dong Li, Hao Wen, Yueheng Zhu, Chao Liu, Meng Yan, and Hongyu Zhang. 2025. CoSEFA: An LLM-Based Programming Assistant for Secure Code Generation via Supervised Co-Decoding. Association for Computing Machinery, New York, NY, USA, 1198–1202. <https://doi.org/10.1145/3696630.3728609>
- [15] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rygGQyrFvH>
- [16] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [17] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825 [cs.CL]* <https://arxiv.org/abs/2310.06825>
- [18] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).
- [19] Xue Jiang, Yihong Dong, Yongding Tao, Huan Yu Liu, Zhi Jin, and Ge Li. 2025. ROCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 334–346. doi:10.1109/ICSE55347.2025.00133
- [20] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [21] Terry Koo, Frederick Liu, and Luheng He. 2024. Automata-based constraints for language model decoding. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=BDBdblmYz>
- [22] Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard Socher, and Nazneen Rajani. 2021. GeDi: Generative Discriminator Guided Sequence Generation. <https://openreview.net/forum?id=TJSofuZED1B>
- [23] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [24] Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. 2025. CodeUpdateArena: Benchmarking Knowledge Editing on API Updates. *arXiv:2407.06249 [cs.CL]* <https://arxiv.org/abs/2407.06249>
- [25] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [26] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proceedings of the ACM on Programming Languages* 9, PLDI (2025), 601–626.
- [27] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-aligned decoding. *Advances in Neural Information Processing Systems* 37 (2024), 24547–24568.
- [28] Kanghee Park, Timothy Zhou, and Loris D'Antoni. 2025. Flexible and Efficient Grammar-Constrained Decoding. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=L6CYAZpO1k>
- [29] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. SynchroMesh: Reliable Code Generation from Pre-trained Language Models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=KmtVD97J43e>
- [30] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [31] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [32] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. doi:10.18653/v1/2021.emnlp-main.779
- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [34] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2025. LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-based Code Completion. *arXiv:2406.09834 [cs.SE]* <https://arxiv.org/abs/2406.09834>
- [35] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.
- [36] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [37] Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing, Yuan-Fang Li, and Gholamreza Haffari. 2024. VersiCode: Towards Version-controllable Code Generation. *arXiv:2406.07411 [cs.SE]* <https://arxiv.org/abs/2406.07411>
- [38] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: Continual Pre-training on Sketches for Library-oriented Code Generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, Lud De Raedt

- (Ed.). International Joint Conferences on Artificial Intelligence Organization, 2369–2375. doi:10.24963/ijcai.2022/329 Main Track.
- [39] Kechi Zhang, Huangzhao Zhang, Ge Li, Jia Li, Zhuo Li, and Zhi Jin. 2023. Tool-coder: Teach code generation models to use api search tools. *arXiv preprint arXiv:2305.04032* (2023).
- [40] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=Lr8cOOtYbfl>.